You said:

Air_Hockey-main(1...
File

Lab_4_VibeCoding...
PDF

give me the 4 tasks implementations codes required in this task by task

ChatGPT said:

I checked your `Air_Hockey-main` project and its README. Your Lab 4 has **exactly 4 implementation tasks**. The project uses Python and Pygame, and the changes need to be made to the existing code rather than building a new game from scratch.

The four tasks are:

1. Fix puck–paddle collision physics.
2. Implement match scoring and winner determination.
3. Add a 30-second countdown timer and draw detection.
4. Reset the puck properly after every goal.

I've prepared an updated version of your project with all four tasks implemented.

Download the completed Air Hockey project ZIP ↗

Task 1 — Fix puck–paddle collision

File: `air-hockey/game/collisions.py`

The original code simply reversed the puck's horizontal velocity when it touched a paddle. This caused incorrect rebounds at angles and could leave the puck stuck inside a paddle.

Replace the contents of `collisions.py` with:

```
"""Reliable puck and paddle collision handling."""
import math


def handle_paddle_collision(puck, paddle):
    dx = puck.x - paddle.x
    dy = puck.y - paddle.y
    distance = math.hypot(dx, dy)
    min_distance = puck.radius + paddle.radius
```

```
    if distance >= min_distance:
        return False

    if distance == 0:
        dx, dy, distance = 1.0, 0.0, 1.0

    nx = dx / distance
    ny = dy / distance

    # Move the puck outside the paddle.
    overlap = min_distance - distance
    puck.x += nx * overlap
    puck.y += ny * overlap

    # Reflect velocity along the collision normal.
    velocity = puck.vx * nx + puck.vy * ny

    if velocity < 0:
        puck.vx -= 2 * velocity * nx
        puck.vy -= 2 * velocity * ny

    # Prevent the puck from becoming too slow.
    speed = math.hypot(puck.vx, puck.vy)

    if speed < 4.0:
        if speed == 0:
            puck.vx = nx * 4.0
            puck.vy = ny * 4.0
        else:
            puck.vx *= 4.0 / speed
            puck.vy *= 4.0 / speed

    return True
```

What this implements: angle-based rebounds, overlap correction, and prevention of repeated collision responses when the puck is already moving away from the paddle.

Task 2 — Implement match scoring

File: `air-hockey/game/game_engine.py`

The game needs two score variables. A goal on the left awards a point to the computer; a goal on the right awards a point to the player.

Add these initial values to `GameEngine` initialization:

```
self.player_score = 0
self.computer_score = 0
```

Use the following goal-handling method:

```
def _handle_goals(self):
    from game.renderer import WIDTH

    if self.puck.x + self.puck.radius < MARGIN:
        if GOAL_TOP + self.puck.radius <= self.puck.y <= GOAL_BOTTOM -
self.puck.radius:
            self.computer_score += 1
            self._reset_puck()
        else:
            self.puck.x = MARGIN + self.puck.radius
            self.puck.vx = abs(self.puck.vx)

    elif self.puck.x - self.puck.radius > WIDTH - MARGIN:
        if GOAL_TOP + self.puck.radius <= self.puck.y <= GOAL_BOTTOM -
self.puck.radius:
            self.player_score += 1
            self._reset_puck()
        else:
            self.puck.x = WIDTH - MARGIN - self.puck.radius
            self.puck.vx = -abs(self.puck.vx)
```

To display the scores, add the following lines inside `draw()` in `game_engine.py`, after drawing the puck:

```
renderer.draw_text(
    surface, font, f"You: {self.player_score}", (30, 28)
)

renderer.draw_text(
    surface, font, f"Computer: {self.computer_score}",
    (WIDTH - 220, 28)
)
```

This also ensures that contact with the wall outside the goal opening results in a bounce rather than a score.

Task 3 — Add a 30-second match timer

Files: `air-hockey/game/game_engine.py` and `air-hockey/main.py`

A. Add the timer variables

At the top of `game_engine.py`, add:

```
MATCH_LENGTH = 30.0
```

Initialize the match timer and result state when starting or restarting the game:

```
self.time_remaining = MATCH_LENGTH
self.game_over = False
self.result_text = ""
```

B. Implement the countdown and match result

Add these methods to `GameEngine`:

```
def update(self, dt=1 / 60):
    if self.game_over:
        return

    self.time_remaining = max(
        0.0, self.time_remaining - max(0.0, dt)
    )

    if self.time_remaining <= 0:
        self._finish_match()
        return

    self.ai.update(self.computer, self.puck)
    self.puck.move()
    self.puck.bounce_off_walls(HEIGHT, MARGIN)

    handle_paddle_collision(self.puck, self.player)
    handle_paddle_collision(self.puck, self.computer)
```

```
    self._handle_goals()


def _finish_match(self):
    self.game_over = True
    self.puck.vx = 0.0
    self.puck.vy = 0.0

    if self.player_score > self.computer_score:
        self.result_text = "You win!"
    elif self.computer_score > self.player_score:
        self.result_text = "Computer wins!"
    else:
        self.result_text = "Draw!"
```

C. Display the remaining time and result

Add these lines to the `draw()` method:

```
renderer.draw_text(
    surface, font,
    f"Time: {self.time_remaining:04.1f}s",
    (WIDTH // 2 - 75, 28)
)

if self.game_over:
    renderer.draw_banner(
        surface, font,
        f"{self.result_text}  Press R to restart"
    )
```

D. Update `main.py`

The timer must receive the actual elapsed time between frames. Replace the relevant game loop code with:

```
dt = clock.tick(60) / 1000.0

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False
```

```
    elif event.type == pygame.KEYDOWN and event.key == pygame.K_r:
        engine.reset_match()

keys = pygame.key.get_pressed()
engine.handle_input(keys)
engine.update(dt)
engine.draw(screen, font)

pygame.display.flip()
```

The match stops when the timer reaches zero, and the game displays the winner or `Draw!` when the scores are equal. Pressing **R** starts a new match.

Task 4 — Reset the puck after every goal

File: `air-hockey/game/game_engine.py`

The original `_reset_puck()` placed the puck at the center but set both velocities to zero. As a result, the puck could remain motionless after a goal.

Replace `_reset_puck()` with:

```
def _reset_puck(self):
    self.puck.x = WIDTH / 2
    self.puck.y = HEIGHT / 2

    self.puck.vx = 0.0
    self.puck.vy = 0.0

    self._launch_puck()
```

Make sure `_launch_puck()` initializes a fresh velocity and direction:

```
def _launch_puck(self):
    angle = random.choice([-0.65, -0.4, 0.4, 0.65])
    direction = random.choice([-1, 1])

    self.puck.vx = INITIAL_PUCK_SPEED * direction
    self.puck.vy = INITIAL_PUCK_SPEED * angle
```

The existing goal-handling method should call `self._reset_puck()` immediately after increasing the appropriate score.

This resets the puck's position and gives it a new velocity for the next point.

Final implementation checklist

- **Task 1 — Collision physics**
  `game/collisions.py` — angle-based rebounds and overlap correction.
- **Task 2 — Scoring**
  `game/game_engine.py` and `game/renderer.py` — score tracking and on-screen display.
- **Task 3 — Match timer**
  `game/game_engine.py` and `main.py` — 30-second countdown, match end, winner or draw, and restart.
- **Task 4 — Puck reset**
  `game/game_engine.py` — center positioning and fresh puck velocity after goals.

How to run the completed project

1. Download and extract the completed project ZIP ↗.
2. Open the `air-hockey` folder in VS Code.
3. Open the terminal in that folder and run:

```
pip install -r requirements.txt
python main.py
```

4. Test all four tasks: hit the puck at different angles, score goals, observe the countdown, and verify that the puck launches again after each goal.
5. Let the timer reach zero to verify the final result. Press **R** to restart.

Important for your submission: The ZIP contains the integrated implementation, not a replacement for your required before-and-after recordings. Keep your original starter project or initial recording, record the completed gameplay separately, and upload the updated code

and your complete ChatGPT conversation link as required by the README.